

PROJECT BASED LEARNING REPORT



RESIDENTIAL SOCIETY RESIDENT AND MAINTENANCE BILLING SYSTEM

A Database Management Systems Project Report

Submitted in partial fulfilment of the requirements of the course

SUBMITTED BY

Name: Mohammad Saalim

Roll Number: 25WU0102159

Programme: B.Tech Computer Science and Engineering (AI & ML)

Section: Panthers

Course: Database Management Systems

Faculty Name: Dr. Kiran Mayee

Academic Year: 2026 – 2027

ABSTRACT

Housing societies and apartment communities collect a monthly maintenance charge from every flat to pay for security, housekeeping, water supply, lift operation, common electricity and repairs. In many societies this work is still recorded in paper registers or scattered spreadsheets, which makes it difficult to know who lives in which flat, which bills have been issued, which payments have been received and which residents are in arrears. This project presents the design and implementation of a Residential Society Resident and Maintenance Billing System built on a relational database.

The system stores its data in six related tables: Society, Block, Flat, Resident, Maintenance_Bill and Payment. These were derived from an Entity-Relationship (ER) model, mapped to a relational schema and verified against the first, second and third normal forms. Primary keys, foreign keys, unique, check and not-null constraints protect data integrity. Two triggers prevent overpayment and keep the status of every bill (Paid, Partial or Unpaid) consistent with the payments received, and a view reports the balance outstanding on each bill.

The database was populated with sample data for one society with three blocks, twelve flats, twelve residents, thirty-three monthly bills and twenty-one payments. Ten SQL queries, including joins, aggregation, subqueries and the query presented in Presentation-II, generate reports such as defaulter lists, block-wise collection and month-wise dues. A simple web interface developed with Python, Flask and SQLite allows the society office to view residents and bills, record payments and see defaulters. Nineteen test cases covering constraints, triggers and web pages were executed and all of them passed. This report documents the requirements, design, implementation and testing of the system and proposes enhancements such as online payment and automated reminders.

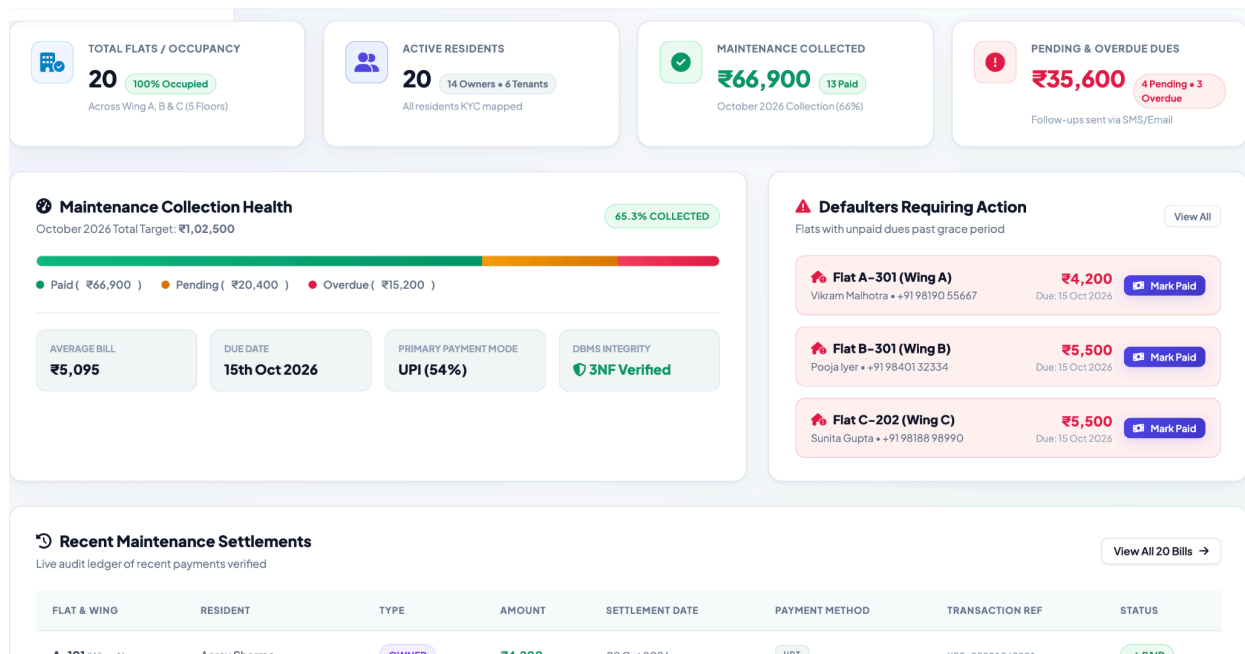
Keywords: DBMS, ER model, normalization, SQL, triggers, maintenance billing, housing society.

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

A residential society is a community of flats grouped into blocks and managed by a managing committee or a hired facility manager. Every flat owner or tenant is expected to pay a monthly maintenance charge, which the society uses to meet the cost of shared services. The amount is usually decided from the size of the flat, with additional charges for water, parking and any penalty for late payment. The society office must therefore keep four kinds of information at all times: the structure of the society (blocks and flats), the people who live in each flat, the bills raised every month and the payments received against those bills.

When this information is kept in notebooks or in several unconnected spreadsheets, the same detail is written many times, figures are copied by hand and the records drift out of agreement. A database management system (DBMS) solves this problem by keeping each fact in exactly one place, by relating the facts through keys and by letting the user ask questions of the data through a standard query language, SQL [1], [2]. This project applies the relational model [3] and the Entity-Relationship approach [4] to the maintenance-billing problem of a residential society.



1.2 Problem Statement

The accounts of a typical society are maintained manually. This causes the following problems:

- **Scattered records.** Resident details, flat details and payment receipts are stored in different registers, so a complete picture of one flat takes a long time to assemble.
- **Calculation errors.** The total bill of a flat (maintenance, water, parking and late fee) is added by hand every month and mistakes are common.
- **No reliable defaulter list.** There is no quick way to find which residents have unpaid or partly paid bills, or how much each one owes.
- **Duplicate and inconsistent data.** A resident's phone number or flat details may be recorded differently in different places.
- **Weak control over payments.** A receipt can be entered twice, entered against the wrong bill or entered for more than the amount due.
- **Poor reporting.** Block-wise collection, month-wise dues and payment-method summaries must be worked out manually for every committee meeting.

The problem addressed by this project is to design and implement a relational database, together with a simple user interface, that records residents, flats and blocks, generates monthly maintenance bills, records payments against bills, keeps the status of each bill correct automatically, and answers the reporting questions of the society office accurately and quickly.

1.3 Proposed Solution

The proposed system stores all society data in a normalized relational database with six tables. Integrity constraints reject invalid data at the point of entry. Triggers apply the business rules for payments inside the database itself, so that the rules hold no matter which program or user changes the data. SQL queries and a view provide the reports, and a lightweight web interface gives the office staff a convenient way to use the system without writing SQL.

All Bills 20Paid 13Pending 4Overdue 3

Search flat, resident name, txn ID...October 2026

BILL ID	FLAT NO	RESIDENT NAME	RESIDENT TYPE	BILLING MONTH	AMOUNT (₹)	DUE DATE	STATUS	PAYMENT MODE	TRANSACTION REF	ACTIONS
#B-001	A-101 (Wing A)	Aarav Sharma	OWNER	2026-10	₹4,200	14T18:30:00.000Z Oct 2026	PAID	UPI	UPI-99281048291	✓ Cleared
#B-002	A-102 (Wing A)	Priya Nair	TENANT	2026-10	₹4,200	14T18:30:00.000Z Oct 2026	PAID	NET_BANKING	HDFC-TXN-772819	✓ Cleared
#B-003	A-201 (Wing A)	Rohan Mehta	OWNER	2026-10	₹5,500	14T18:30:00.000Z Oct 2026	PENDING	-	-	Mark Paid
#B-004	A-202 (Wing A)	Ananya Deshmukh	OWNER	2026-10	₹5,500	14T18:30:00.000Z Oct 2026	PAID	CREDIT_CARD	CC-AUTH-882910	✓ Cleared
#B-005	A-301 (Wing A)	Vikram Malhotra	TENANT	2026-10	₹4,200	14T18:30:00.000Z Oct 2026	OVERDUE	-	-	Mark Paid
#B-006	A-302 (Wing A)	Kavita Singhania	OWNER	2026-10	₹5,500	14T18:30:00.000Z Oct 2026	PAID	UPI	UPI-88192019283	✓ Cleared
#B-007	A-401 (Wing A)	Siddharth Verma	OWNER	2026-10	₹7,000	14T18:30:00.000Z Oct 2026	PAID	NET_BANKING	SBI-NEFT-991823	✓ Cleared
#B-008	B-101 (Wing B)	Neha Patel	TENANT	2026-10	₹3,200	14T18:30:00.000Z Oct 2026	PENDING	-	-	Mark Paid

2. OBJECTIVES AND SCOPE

2.1 Objectives

The main objectives of the project are:

1. To model the society, its blocks, flats, residents, monthly bills and payments using an Entity-Relationship diagram.
2. To convert the ER diagram into a relational schema and normalize it up to the third normal form (3NF) so that redundancy and update anomalies are avoided.
3. To define data types, primary keys, foreign keys and check, unique and not-null constraints that keep the data accurate.
4. To store the monthly maintenance bill of every flat with its component charges (maintenance, water, parking and late fee) and its due date.
5. To record payments, including part payments, and to update the status of the bill automatically using triggers.
6. To prepare SQL queries that answer the common questions of the society office, such as defaulters, collections and vacant flats.
7. To provide a simple, plain web interface for viewing records, recording payments and viewing defaulters.
8. To test the database constraints, triggers and interface with documented test cases.

2.2 Scope of the System

The system covers the following functions:

- Maintaining master data for the society, blocks and flats.
- Maintaining resident records, including owner or tenant type and move-in date, with one or more residents for each flat.
- Storing one maintenance bill per flat per month and applying late fees to overdue bills.
- Recording payments by cash, UPI, card or net banking, with a transaction reference.
- Reporting the balance and status of every bill, the defaulter list and summaries by block, month and payment method.

2.3 Limitations

The present version is an academic project with these limitations: it is a single-office application without user login or role-based access; it has no online payment gateway, so the office enters payments after the money is received; it sends no SMS or e-mail reminders; and monthly bills are loaded as data rather than generated from the interface. These are listed as future enhancements in Chapter 12.

3. SOFTWARE AND HARDWARE REQUIREMENTS

The system was developed and tested on the software and hardware listed below. The SQL scripts use standard SQL and can be run on other relational databases such as MySQL with minor changes to data types and the syntax of triggers [11].

3.1 Software Requirements

Table 3.1: Software requirements

Component	Specification
Operating system	Windows 10/11, Linux or macOS (developed and tested on Ubuntu 24.04 Linux)
Database management system	SQLite 3.45 (embedded relational database)
Programming language	Python 3.12
Web framework	Flask 3.1
Database connectivity	Python sqlite3 module (DB-API 2.0)
Front end	HTML 5 and CSS, rendered by any modern web browser
Editor / IDE	Any text editor or IDE, for example Visual Studio Code
Version control	Git and GitHub

3.2 Hardware Requirements

Table 3.2: Hardware requirements

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or equivalent
Memory (RAM)	4 GB	8 GB
Free disk space	500 MB	1 GB
Display	1366 x 768 pixels	1920 x 1080 pixels
Network	Not required for a single computer	LAN or Internet for multiple office users

4. ER DIAGRAM

The Entity-Relationship (ER) diagram in Figure 4.1 models the data of the system. It contains six entities (Society, Block, Flat, Resident, Maintenance_Bill and Payment) and five one-to-many relationships. Primary keys are underlined and foreign keys are marked FK. The notation “1” denotes one and “N” denotes many.

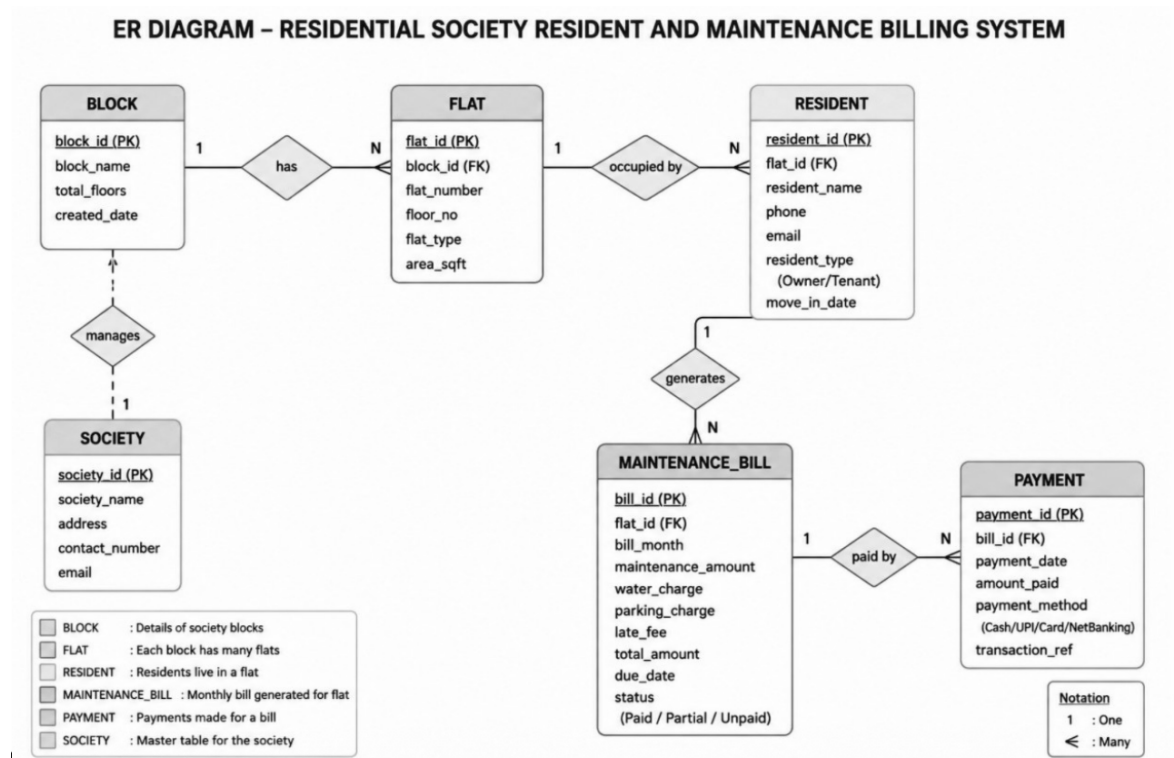


Figure 4.1: ER diagram of the Residential Society Resident and Maintenance Billing System

4.1 Entities

Table 4.1: Entities and their purpose

Entity	Primary key	Purpose
SOCIETY	society_id	Master table holding the name, address and contact details of the society.
BLOCK	block_id	A block or tower of the society, with its number of floors.
FLAT	flat_id	A flat in a block, with its floor, type (1BHK, 2BHK, 3BHK) and area.
RESIDENT	resident_id	A person living in a flat, either as an owner or as a tenant.
MAINTENANCE_BILL	bill_id	The monthly bill raised for a flat, with its component charges, due date and status.
PAYMENT	payment_id	A payment received against a bill, with its date, amount, method and reference.

4.2 Relationships and Cardinality

Table 4.2: Relationships in the ER diagram

Relationship	Entities	Cardinality	Meaning
manages	SOCIETY – BLOCK	1 : N	A society manages many blocks; each block belongs to one society.
has	BLOCK – FLAT	1 : N	A block has many flats; each flat is in exactly one block.
occupied by	FLAT – RESIDENT	1 : N	A flat can be occupied by many residents; each resident lives in one flat.
generates	RESIDENT – MAINTENANCE_BILL	1 : N	Bills are generated for the household; each bill is stored against a flat_id.
paid by	MAINTENANCE_BILL – PAYMENT	1 : N	A bill can be settled by one or more payments; each payment belongs to one bill.

The diagram lists flat_id as the foreign key of MAINTENANCE_BILL, so in the implementation a bill is raised for a flat, which is the unit that is actually charged. The resident or residents responsible for a bill are identified through the occupied-by relationship between the flat and its residents. The BLOCK entity in the diagram does not list a society_id attribute; when the manages relationship was mapped to tables, the society_id foreign key was placed in the BLOCK table, as described in Chapter 5.

5. RELATIONAL SCHEMA AND NORMALIZATION

5.1 Mapping the ER Diagram to Relations

The ER diagram was converted to a relational schema using the standard mapping rules [1]. Each entity became a table and each attribute became a column. For every one-to-many relationship, the primary key of the “one” side was added to the “many” side as a foreign key. Because every relationship in this design is one-to-many, no separate relationship table was needed.

Table 5.1: Mapping of relationships to foreign keys

Relationship	Foreign key added	Referenced table
manages	BLOCK.society_id	SOCIETY(society_id)
has	FLAT.block_id	BLOCK(block_id)
occupied by	RESIDENT.flat_id	FLAT(flat_id)
generates	MAINTENANCE_BILL.flat_id	FLAT(flat_id)
paid by	PAYMENT.bill_id	MAINTENANCE_BILL(bill_id)

5.2 Relational Schema

The resulting relational schema is shown below. PK denotes the primary key and FK denotes a foreign key.

SOCIETY (society_id PK, society_name, address, contact_number, email)
BLOCK (block_id PK, society_id FK → SOCIETY, block_name, total_floors, created_date)
FLAT (flat_id PK, block_id FK → BLOCK, flat_number, floor_no, flat_type, area_sqft)
RESIDENT (resident_id PK, flat_id FK → FLAT, resident_name, phone, email, resident_type, move_in_date)
MAINTENANCE_BILL (bill_id PK, flat_id FK → FLAT, bill_month, maintenance_amount, water_charge, parking_charge, late_fee, total_amount, due_date, status)
PAYMENT (payment_id PK, bill_id FK → MAINTENANCE_BILL, payment_date, amount_paid, payment_method, transaction_ref)

5.3 Normalization

Normalization is the process of organizing the columns of a database into tables so that redundancy is removed and anomalies in insertion, deletion and updating are avoided [1], [5]. The design was reached by starting from the single sheet that a society office would keep in a register and normalizing it step by step.

Skyline Heights
SOCIETY MANAGEMENT

Flats & Residential Towers
Real-time occupancy, billing audits & resident records (20 Verified Units)

Dark Mode | October 2026 Cycle | Export Audit | New Resident

Society Wings & Units Layout
20 Total Flats mapped across 3 Residential Towers

Legend: Paid (Green), Pending (Orange), Overdue (Red)

Wing A
7 Occupied Units • Floor 1 to 5
100% OCCUPANCY

Unit	BHK	Sqft	Resident	Status
A-101	2BHK	1150	Aarav Sharma	PAID
A-102	2BHK	1150	Priya Nair	PAID
A-201	3BHK	1550	Rohan Mehta	PAID
A-202	3BHK	1600	Ananya Deshmukh	PAID
A-301	2BHK	1150	Vikram Malhotra	PAID
A-302	3BHK	1550	Kavita Singhania	PAID
A-401	4BHK	2200	Siddharth Verma	PAID

Wing B
7 Occupied Units • Floor 1 to 5
100% OCCUPANCY

Unit	BHK	Sqft	Resident	Status
B-101	1BHK	750	Neha Patel	PAID
B-102	2BHK	1180	Aditya Rao	PAID
B-201	2BHK	1200	Sneha Kulkarni	PAID
B-202	3BHK	1620	Rajesh Bhatia	PAID
B-301	3BHK	1500	Pooja Iyer	PAID
B-302	2BHK	1200	Sameer Chopra	PAID
B-401	Penthouse			PAID

Wing C
6 Occupied Units • Floor 1 to 5
100% OCCUPANCY

Unit	BHK	Sqft	Resident	Status
C-101	1BHK	760	Amit Joshi	PAID
C-102	2BHK	1190	Deepika Menon	PAID
C-201	2BHK	1210	Harsh Vardhan	PAID
C-202	3BHK	1650	Sunita Gupta	PAID
C-301	3BHK	1600	Karan Kapoor	PAID
C-401	Penthouse	2900	Meera Sen	PAID

MySQL 8.x Active
root@127.0.0.1/residential_so-
MySQL Settings

5.3.1 Unnormalized Form (UNF)

The office sheet records everything about a bill in one row. Curly braces mark groups that repeat:

```
BILLING_SHEET (society_name, society_address, society_contact, block_name,  
total_floors, flat_number, floor_no, flat_type, area_sqft,  
{ resident_name, phone, email, resident_type, move_in_date },  
bill_month, maintenance_amount, water_charge, parking_charge, late_fee,  
total_amount, due_date, status,  
{ payment_date, amount_paid, payment_method, transaction_ref })
```

This form has several problems. The society name and address are repeated on every row. A flat's type and area are repeated for every month. A flat with two residents or a bill with two payments needs repeating groups. Changing a block name requires editing many rows (update anomaly), a flat cannot be recorded until a bill exists for it (insertion anomaly), and deleting the only bill of a flat also deletes the flat's details (deletion anomaly).

5.3.2 First Normal Form (1NF)

A relation is in 1NF when every column holds a single atomic value and there are no repeating groups. The repeating groups of residents and payments were removed by giving each resident and each payment its own row, and every row was given a unique identifier. All columns now hold atomic values; for example, the address is stored as one value and the bill month as a single YYYY-MM value.

5.3.3 Second Normal Form (2NF)

A relation is in 2NF when it is in 1NF and no non-key column depends on only part of a composite key. In the 1NF sheet the key combines the flat, the bill month, the resident and the payment, and several columns depend on only a part of it: flat_type and area_sqft depend only on the flat; resident_name, phone and resident_type depend only on the resident; the charge columns, due_date and status depend only on the flat and the month; and payment_date, amount_paid and payment_method depend only on the payment. Removing these partial dependencies separated the data into the FLAT, RESIDENT, MAINTENANCE_BILL and PAYMENT relations, each with a single-column surrogate primary key.

5.3.4 Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and no non-key column depends on another non-key column. In the FLAT relation, block_name and total_floors depend on block_id and not on the flat itself, which is a transitive dependency. In the BLOCK relation, society_name, address and contact_number depend on the society. These attributes were moved into their own relations, BLOCK and SOCIETY, and are linked back by foreign keys. The result is the six relations given in Section 5.2.

5.3.5 Verification of the Final Design

Table 5.2 lists the candidate keys and the functional dependencies of each final relation. In every relation, every determinant is a candidate key, so the relations also satisfy Boyce–Codd Normal Form (BCNF) with respect to the dependencies listed.

Table 5.2: Candidate keys and functional dependencies

Relation	Candidate keys	Functional dependencies
SOCIETY	society_id; society_name; email	society_id $\rightarrow$ society_name, address, contact_number, email
BLOCK	block_id; (society_id, block_name)	block_id $\rightarrow$ society_id, block_name, total_floors, created_date
FLAT	flat_id; (block_id, flat_number)	flat_id $\rightarrow$ block_id, flat_number, floor_no, flat_type, area_sqft
RESIDENT	resident_id; phone	resident_id $\rightarrow$ flat_id, resident_name, phone, email, resident_type, move_in_date
MAINTENANCE_BILL	bill_id; (flat_id, bill_month)	bill_id $\rightarrow$ all other columns
PAYMENT	payment_id; transaction_ref	payment_id $\rightarrow$ bill_id, payment_date, amount_paid, payment_method, transaction_ref

Two columns of MAINTENANCE_BILL are derived from other data: total_amount equals the sum of the four charge columns, and status reflects the payments received. They are stored deliberately. total_amount records the amount that was actually billed, so that later changes to rates do not alter an old bill, and a CHECK constraint guarantees that it always equals the sum of its components. status is maintained automatically by a trigger, which prevents it from disagreeing with the payments. This is a controlled and documented use of derived data and does not introduce update anomalies.

6. DATA DICTIONARY

The data dictionary describes every column of every table with its data type, constraints and meaning. Standard SQL type names are used. In SQLite, which was used for the implementation, these types map to the INTEGER, TEXT and NUMERIC storage classes, and dates are held as text in the format YYYY-MM-DD.

6.1 Table: SOCIETY

Table 6.1: Structure of SOCIETY

Column	Data type	Constraints	Description
society_id	INT	PRIMARY KEY, AUTOINCREMENT	Unique identifier of the society
society_name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the society
address	VARCHAR(200)	NOT NULL	Postal address
contact_number	VARCHAR(15)	NOT NULL	Society office telephone number
email	VARCHAR(100)	UNIQUE	Society office e-mail address

6.2 Table: BLOCK

Table 6.2: Structure of BLOCK

Column	Data type	Constraints	Description
block_id	INT	PRIMARY KEY, AUTOINCREMENT	Unique identifier of the block
society_id	INT	NOT NULL, FOREIGN KEY → society(society_id), ON DELETE RESTRICT	Society to which the block belongs
block_name	VARCHAR(30)	NOT NULL, UNIQUE with society_id	Name of the block, for example Block A
total_floors	INT	NOT NULL, CHECK (total_floors > 0)	Number of floors in the block
created_date	DATE	NOT NULL, DEFAULT current date	Date on which the block record was created

6.3 Table: FLAT

Table 6.3: Structure of FLAT

Column	Data type	Constraints	Description
flat_id	INT	PRIMARY KEY, AUTOINCREMENT	Unique identifier of the flat
block_id	INT	NOT NULL, FOREIGN KEY → block(block_id), ON DELETE RESTRICT	Block in which the flat is located
flat_number	VARCHAR(10)	NOT NULL, UNIQUE with block_id	Flat number, for example A-101
floor_no	INT	NOT NULL, CHECK (floor_no >= 0)	Floor on which the flat is located
flat_type	VARCHAR(10)	NOT NULL, CHECK IN ('1BHK','2BHK','3BHK')	Type of flat
area_sqft	INT	NOT NULL, CHECK (area_sqft > 0)	Built-up area in square feet

6.4 Table: RESIDENT

Table 6.4: Structure of RESIDENT

Column	Data type	Constraints	Description
resident_id	INT	PRIMARY KEY, AUTOINCREMENT	Unique identifier of the resident
flat_id	INT	NOT NULL, FOREIGN KEY → flat(flat_id), ON DELETE RESTRICT	Flat occupied by the resident
resident_name	VARCHAR(100)	NOT NULL	Full name of the resident
phone	VARCHAR(15)	NOT NULL, UNIQUE	Mobile number of the resident
email	VARCHAR(100)	NULL allowed	E-mail address of the resident
resident_type	VARCHAR(10)	NOT NULL, CHECK IN ('Owner','Tenant')	Whether the resident is an owner or a tenant
move_in_date	DATE	NOT NULL	Date on which the resident moved in

6.5 Table: MAINTENANCE_BILL

Table 6.5: Structure of MAINTENANCE_BILL

Column	Data type	Constraints	Description
bill_id	INT	PRIMARY KEY, AUTOINCREMENT	Unique identifier of the bill
flat_id	INT	NOT NULL, FOREIGN KEY → flat(flat_id), ON DELETE RESTRICT	Flat for which the bill is raised
bill_month	CHAR(7)	NOT NULL, UNIQUE with flat_id	Billing month in the format YYYY-MM
maintenance_amount	DECIMAL(10,2)	NOT NULL, CHECK (≥ 0)	Monthly maintenance charge
water_charge	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (≥ 0)	Water charge for the month
parking_charge	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (≥ 0)	Parking charge for the month
late_fee	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (≥ 0)	Penalty added for late payment
total_amount	DECIMAL(10,2)	NOT NULL, CHECK (total = sum of the four charges)	Total amount billed
due_date	DATE	NOT NULL	Last date for payment without penalty
status	VARCHAR(10)	NOT NULL, DEFAULT 'Unpaid', CHECK IN ('Paid','Partial','Unpaid')	Payment status of the bill

6.6 Table: PAYMENT

Table 6.6: Structure of PAYMENT

Column	Data type	Constraints	Description
payment_id	INT	PRIMARY KEY, AUTOINCREMENT	Unique identifier of the payment
bill_id	INT	NOT NULL, FOREIGN KEY → maintenance_bill(bill_id), ON DELETE RESTRICT	Bill against which the payment is made
payment_date	DATE	NOT NULL	Date of payment
amount_paid	DECIMAL(10,2)	NOT NULL, CHECK (amount_paid > 0)	Amount received
payment_method	VARCHAR(12)	NOT NULL, CHECK IN ('Cash','UPI','Card','NetBanking')	Mode of payment
transaction_ref	VARCHAR(40)	UNIQUE	Bank, UPI or receipt reference number

6.7 Other Database Objects

Table 6.7: Indexes, triggers and view

Object	Type	Purpose
idx_flat_block	Index on flat(block_id)	Speeds up joins from block to flat
idx_resident_flat	Index on resident(flat_id)	Speeds up joins from flat to resident
idx_bill_status	Index on maintenance_bill(status)	Speeds up defaulter searches
idx_payment_bill	Index on payment(bill_id)	Speeds up payment totals for a bill
trg_prevent_overpayment	BEFORE INSERT trigger on payment	Rejects a payment that would make the total paid exceed the bill amount
trg_update_bill_status	AFTER INSERT trigger on payment	Sets the bill status to Paid or Partial after each payment
v_bill_balance	View	Shows the amount paid and the balance outstanding for every bill

Skyline Heights
SOCIETY MANAGEMENT

Overview

Maintenance Bills 43

Residents Directory 20

Flats & Towers

DBMS & SQL Console

Flats & Residential Towers

Real-time occupancy, billing audits & resident records (20 Verified Units)

Dark Mode

October 2026 Cycle

Export Audit

New Resident

Society Wings & Units Layout

20 Total Flats mapped across 3 Residential Towers

Wing A

7 Occupied Units • Floor 1 to 5

100% OCCUPANCY

A-101 PAID
2BHK • 1150 sqft
Aarav Sharma

A-102 PAID
2BHK • 1150 sqft
Priya Nair

A-201 PAID
3BHK • 1550 sqft
Rohan Mehta

A-202 PAID
3BHK • 1600 sqft
Ananya Deshmukh

A-301 PAID
2BHK • 1150 sqft
Vikram Malhotra

A-302 PAID
3BHK • 1550 sqft
Kavita Singhania

A-401 PAID
4BHK • 2200 sqft
Siddharth Verma

Wing B

7 Occupied Units • Floor 1 to 5

100% OCCUPANCY

B-101 PAID
1BHK • 750 sqft
Neha Patel

B-102 PAID
2BHK • 1180 sqft
Aditya Rao

B-201 PAID
2BHK • 1200 sqft
Sneha Kulkarni

B-202 PAID
3BHK • 1620 sqft
Rajesh Bhatia

B-301 PAID
3BHK • 1580 sqft
Pooja Iyer

B-302 PAID
2BHK • 1200 sqft
Sameer Chopra

B-401 PAID

Wing C

6 Occupied Units • Floor 1 to 5

100% OCCUPANCY

C-101 PAID
1BHK • 760 sqft
Amit Joshi

C-102 PAID
2BHK • 1190 sqft
Deepika Menon

C-201 PAID
2BHK • 1210 sqft
Harsh Vardhan

C-202 PAID
3BHK • 1650 sqft
Sunita Gupta

C-301 PAID
3BHK • 1600 sqft
Karan Kapoor

C-401 PAID
Penthouse • 2900 sqft
Meera Sen

MySQL 8.x Active

root@127.0.0.1/residential_so...

MySQL Settings

Skyline Heights

SOCIETY MANAGEMENT

Overview

Maintenance Bills43

Residents Directory

20

Flats & Towers

DBMS & SQL Console

Society Residents Directory

Real-time occupancy, billing audits & resident records (20 Verified Units)

Dark Mode

October 2026 Cycle

Export Audit

New Resident

All Residents (20)

Owners (14)

Tenants (6)

Search by name, phone, flat...

Add Resident

RESIDENT ID	FULL NAME	FLAT & WING	FLAT TYPE	TYPE	PHONE	EMAIL	MOVE-IN DATE	EMERGE
#RES-001	Aarav Sharma	A-101 (Wing A)	2BHK (1150 sqft)	OWNER	+91 98201 11223	aarav.sharma@example.com	14T18:30:00.000Z Apr 2021	+91 98201 11223
#RES-002	Priya Nair	A-102 (Wing A)	2BHK (1150 sqft)	TENANT	+91 98450 22334	priya.nair@example.com	31T18:30:00.000Z Jul 2023	+91 98450 22334
#RES-003	Rohan Mehta	A-201 (Wing A)	3BHK (1550 sqft)	OWNER	+91 97110 33445	rohan.mehta@example.com	09T18:30:00.000Z Jan 2020	+91 97110 33445
#RES-004	Ananya Deshmukh	A-202 (Wing A)	3BHK (1600 sqft)	OWNER	+91 99230 44556	ananya.d@example.com	19T18:30:00.000Z Nov 2021	+91 99230 44556
#RES-005	Vikram Malhotra	A-301 (Wing A)	2BHK (1150 sqft)	TENANT	+91 98190 55667	vikram.m@example.com	31T18:30:00.000Z Jan 2024	+91 98190 55667
	Kavita	A-302	3BHK		+91		11T18:30:00.000Z	

MySQL 8.x Active

root@127.0.0.1/residential_so...

MySQL Settings

18

7. SQL COMMANDS USED

This chapter lists the Data Definition Language (DDL) and Data Manipulation Language (DML) statements used in the project, with the output produced when they were executed on the sample database. All statements were executed in SQLite 3.45.

7.1 Data Definition Language (DDL)

The CREATE TABLE statements below define the six tables with their keys and constraints. Foreign key enforcement is switched on with PRAGMA foreign_keys = ON at the start of every database connection.

```
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE society (  
    society_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    society_name VARCHAR(100) NOT NULL UNIQUE,  
    address VARCHAR(200) NOT NULL,  
    contact_number VARCHAR(15) NOT NULL,  
    email VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE block (  
    block_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    society_id INTEGER NOT NULL,  
    block_name VARCHAR(30) NOT NULL,  
    total_floors INTEGER NOT NULL CHECK (total_floors > 0),  
    created_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    UNIQUE (society_id, block_name),  
    FOREIGN KEY (society_id) REFERENCES society(society_id) ON DELETE RESTRICT  
);
```

```
CREATE TABLE flat (  
    flat_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    block_id INTEGER NOT NULL,  
    flat_number VARCHAR(10) NOT NULL,  
    floor_no INTEGER NOT NULL CHECK (floor_no >= 0),  
    flat_type VARCHAR(10) NOT NULL CHECK (flat_type IN ('1BHK','2BHK','3BHK')),  
    area_sqft INTEGER NOT NULL CHECK (area_sqft > 0),  
    UNIQUE (block_id, flat_number),  
    FOREIGN KEY (block_id) REFERENCES block(block_id) ON DELETE RESTRICT  
);
```

```

CREATE TABLE resident (
  resident_id INTEGER PRIMARY KEY AUTOINCREMENT,
  flat_id INTEGER NOT NULL,
  resident_name VARCHAR(100) NOT NULL,
  phone VARCHAR(15) NOT NULL UNIQUE,
  email VARCHAR(100),
  resident_type VARCHAR(10) NOT NULL CHECK (resident_type IN ('Owner','Tenant')),
  move_in_date DATE NOT NULL,
  FOREIGN KEY (flat_id) REFERENCES flat(flat_id) ON DELETE RESTRICT
);

```

```

CREATE TABLE maintenance_bill (
  bill_id INTEGER PRIMARY KEY AUTOINCREMENT,
  flat_id INTEGER NOT NULL,
  bill_month CHAR(7) NOT NULL,
  maintenance_amount DECIMAL(10,2) NOT NULL CHECK (maintenance_amount >= 0),
  water_charge DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (water_charge >= 0),
  parking_charge DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (parking_charge >= 0),
  late_fee DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (late_fee >= 0),
  total_amount DECIMAL(10,2) NOT NULL,
  due_date DATE NOT NULL,
  status VARCHAR(10) NOT NULL DEFAULT 'Unpaid'
  CHECK (status IN ('Paid','Partial','Unpaid')),
  UNIQUE (flat_id, bill_month),
  CHECK (total_amount = maintenance_amount + water_charge + parking_charge + late_fee),
  FOREIGN KEY (flat_id) REFERENCES flat(flat_id) ON DELETE RESTRICT
);

```

```

CREATE TABLE payment (
  payment_id INTEGER PRIMARY KEY AUTOINCREMENT,
  bill_id INTEGER NOT NULL,
  payment_date DATE NOT NULL,
  amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
  payment_method VARCHAR(12) NOT NULL
  CHECK (payment_method IN ('Cash','UPI','Card','NetBanking')),
  transaction_ref VARCHAR(40) UNIQUE,
  FOREIGN KEY (bill_id) REFERENCES maintenance_bill(bill_id) ON DELETE RESTRICT
);

```

Indexes, triggers and the view were created with the following statements:

```

CREATE INDEX idx_flat_block ON flat(block_id);
CREATE INDEX idx_resident_flat ON resident(flat_id);
CREATE INDEX idx_bill_status ON maintenance_bill(status);
CREATE INDEX idx_payment_bill ON payment(bill_id);

```

```
-- Trigger 1: reject a payment that would exceed the bill total
CREATE TRIGGER trg_prevent_overpayment
BEFORE INSERT ON payment
BEGIN
    SELECT RAISE(ABORT, 'Payment exceeds outstanding bill amount')
    WHERE (SELECT COALESCE(SUM(amount_paid), 0) FROM payment WHERE bill_id =
NEW.bill_id)
        + NEW.amount_paid >
        (SELECT total_amount FROM maintenance_bill WHERE bill_id = NEW.bill_id);
END;
```

```
-- Trigger 2: keep bill status in step with payments received
CREATE TRIGGER trg_update_bill_status
AFTER INSERT ON payment
BEGIN
    UPDATE maintenance_bill
    SET status = CASE
        WHEN (SELECT SUM(amount_paid) FROM payment WHERE bill_id = NEW.bill_id) >=
total_amount
        THEN 'Paid' ELSE 'Partial' END
    WHERE bill_id = NEW.bill_id;
END;
```

```
-- View: balance outstanding on every bill
CREATE VIEW v_bill_balance AS
SELECT b.bill_id, b.flat_id, b.bill_month, b.total_amount, b.due_date, b.status,
    COALESCE(SUM(p.amount_paid), 0) AS paid_amount,
    b.total_amount - COALESCE(SUM(p.amount_paid), 0) AS balance
FROM maintenance_bill b
LEFT JOIN payment p ON p.bill_id = b.bill_id
GROUP BY b.bill_id;
```

7.2 Data Manipulation Language (DML)

7.2.1 INSERT

Sample data was inserted into every table. A few rows of each INSERT statement are shown here; the complete script is included in the project repository.

```
INSERT INTO society (society_name, address, contact_number, email) VALUES
('Sunrise Heights Residency', 'Plot 12, Green Park Layout, Hyderabad - 500081', '9876500001',
'office@sunriseheights.example');
```

```
INSERT INTO block (society_id, block_name, total_floors, created_date) VALUES
(1, 'Block A', 4, '2018-04-10'),
(1, 'Block B', 4, '2018-04-10'),
(1, 'Block C', 3, '2019-01-15');
```

```
INSERT INTO flat (block_id, flat_number, floor_no, flat_type, area_sqft) VALUES
(1, 'A-101', 1, '2BHK', 1050),
(1, 'A-102', 1, '3BHK', 1450),
(1, 'A-201', 2, '1BHK', 650),
(1, 'A-202', 2, '2BHK', 1050),
... (8 more rows)
```

```
INSERT INTO resident (flat_id, resident_name, phone, email, resident_type, move_in_date) VALUES
(1, 'Ravi Kumar Reddy', '9876510001', 'ravi.reddy@example.com', 'Owner', '2018-06-01'),
(1, 'Sunita Reddy', '9876510002', 'sunita.reddy@example.com', 'Owner', '2018-06-01'),
(2, 'Anil Sharma', '9876510003', 'anil.sharma@example.com', 'Tenant', '2024-02-15'),
... (9 more rows)
```

```
INSERT INTO maintenance_bill (flat_id, bill_month, maintenance_amount, water_charge, parking_charge,
late_fee, total_amount, due_date, status) VALUES
(1, '2026-08', 2625.00, 300.00, 500.00, 0.00, 3425.00, '2026-08-10', 'Unpaid'),
(2, '2026-08', 3625.00, 350.00, 1000.00, 0.00, 4975.00, '2026-08-10', 'Unpaid'),
(3, '2026-08', 1625.00, 250.00, 0.00, 0.00, 1875.00, '2026-08-10', 'Unpaid'),
... (30 more rows)
```

```
INSERT INTO payment (bill_id, payment_date, amount_paid, payment_method, transaction_ref) VALUES
(1, '2026-08-03', 3425.00, 'UPI', 'UPI202608001'),
(3, '2026-08-04', 1875.00, 'Cash', 'CASH202608002'),
(4, '2026-08-05', 3425.00, 'NetBanking', 'NET202608003'),
... (18 more rows)
```

Inserting into the payment table fires the triggers, which update the status of the related bill automatically.

7.2.2 *SELECT with sample outputs*

```
SELECT * FROM society;
```

Table 7.1: Output – SOCIETY

society_id	society_name	address	contact_number	email
1	Sunrise Heights Residency	Plot 12, Green Park Layout, Hyderabad - 500081	9876500001	office@sunriseheights.example

```
SELECT * FROM block;
```

Table 7.2: Output – BLOCK

block_id	society_id	block_name	total_floors	created_date
1	1	Block A	4	2018-04-10
2	1	Block B	4	2018-04-10
3	1	Block C	3	2019-01-15

```
SELECT flat_id, block_id, flat_number, floor_no, flat_type, area_sqft
FROM flat LIMIT 6;
```

Table 7.3: Output – FLAT (first six rows)

flat_id	block_id	flat_number	floor_no	flat_type	area_sqft
1	1	A-101	1	2BHK	1050
2	1	A-102	1	3BHK	1450
3	1	A-201	2	1BHK	650
4	1	A-202	2	2BHK	1050
5	2	B-101	1	2BHK	1050
6	2	B-102	1	1BHK	650

```
SELECT resident_id, flat_id, resident_name, phone, resident_type, move_in_date
FROM resident LIMIT 5;
```

Table 7.4: Output – RESIDENT (first five rows)

resident_id	flat_id	resident_name	phone	resident_type	move_in_date
1	1	Ravi Kumar Reddy	9876510001	Owner	2018-06-01
2	1	Sunita Reddy	9876510002	Owner	2018-06-01
3	2	Anil Sharma	9876510003	Tenant	2024-02-15
4	3	Meera Iyer	9876510004	Owner	2019-03-20
5	4	Imran Khan	9876510005	Tenant	2025-07-01

```
SELECT bill_id, flat_id, bill_month, maintenance_amount, late_fee,
       total_amount, due_date, status
FROM maintenance_bill
WHERE bill_month = '2026-08' LIMIT 5;
```

Table 7.5: Output – MAINTENANCE_BILL (August 2026, first five rows)

bill_id	flat_id	bill_month	maintenance_amount	late_fee	total_amount	due_date	status
1	1	2026-08	2625.00	0.00	3425.00	2026-08-10	Paid
2	2	2026-08	3625.00	150.00	5125.00	2026-08-10	Unpaid
3	3	2026-08	1625.00	0.00	1875.00	2026-08-10	Paid
4	4	2026-08	2625.00	0.00	3425.00	2026-08-10	Paid

bill_id	flat_id	bill_month	maintenance_amount	late_fee	total_amount	due_date	status
5	5	2026-08	2625.00	0.00	3425.00	2026-08-10	Paid

```
SELECT * FROM payment LIMIT 5;
```

Table 7.6: Output – PAYMENT (first five rows)

payment_id	bill_id	payment_date	amount_paid	payment_method	transaction_ref
1	1	2026-08-03	3425.00	UPI	UPI202608001
2	3	2026-08-04	1875.00	Cash	CASH202608002
3	4	2026-08-05	3425.00	NetBanking	NET202608003
4	5	2026-08-06	3425.00	Card	CAR202608004
5	6	2026-08-07	1875.00	UPI	UPI202608005

7.2.3 UPDATE, ALTER and DELETE

The following statements were executed on a working copy of the database to demonstrate modification, structural change and deletion. Deleting a block that still has flats is refused because of the foreign key constraint with ON DELETE RESTRICT.

```
INSERT INTO resident (flat_id, resident_name, phone, email, resident_type, move_in_date)
VALUES (12, 'Neha Joshi', '9876510013', 'neha.joshi@example.com', 'Tenant', '2026-10-01');
```

Result: OK, 1 row(s) affected.

```
UPDATE resident SET phone = '9876599999' WHERE resident_id = 3;
```

Result: OK, 1 row(s) affected.

Table 7.8: Output after the update statement

resident_id	resident_name	phone
3	Anil Sharma	9876599999

```
UPDATE maintenance_bill
SET late_fee = 150,
    total_amount = maintenance_amount + water_charge + parking_charge + 150
WHERE status <> 'Paid' AND due_date < '2026-10-01' AND late_fee = 0;
```

Result: OK, 6 row(s) affected.

```
ALTER TABLE resident ADD COLUMN alternate_phone VARCHAR(15);
```

Result: OK.

DELETE FROM resident WHERE resident_id = 13;
--

Result: OK, 1 row(s) affected.

Table 7.11: Output after the delete statement

residents_remaining
12

DELETE FROM block WHERE block_id = 1;

Result: Error: FOREIGN KEY constraint failed.

The first UPDATE changes a resident's phone number. The second UPDATE applies a late fee of Rs. 150 to all overdue bills that are not paid, and recomputes the total of each such bill in the same statement so that the CHECK constraint remains satisfied. The ALTER TABLE statement adds an optional column without affecting existing rows.

8. QUERIES WITH OUTPUTS

Ten SQL queries were written to answer the reporting needs of the society office. Each query below is shown with its output on the sample database. Amounts are in Indian rupees.

Q1. Residents with their flat number, block and resident type (three-table join)

Joins three tables to list every resident with the flat and block in which the resident lives. It is the basic resident register of the society.

```
SELECT r.resident_name, f.flat_number, b.block_name, r.resident_type
FROM resident r
JOIN flat f ON f.flat_id = r.flat_id
JOIN block b ON b.block_id = f.block_id
ORDER BY b.block_name, f.flat_number;
```

Table 8.1: Output of Query 1

resident_name	flat_number	block_name	resident_type
Ravi Kumar Reddy	A-101	Block A	Owner
Sunita Reddy	A-101	Block A	Owner
Anil Sharma	A-102	Block A	Tenant
Meera Iyer	A-201	Block A	Owner
Imran Khan	A-202	Block A	Tenant
Priya Nair	B-101	Block B	Owner
Suresh Babu	B-102	Block B	Owner
Kavita Desai	B-201	Block B	Tenant
Rahul Verma	B-202	Block B	Owner
Lakshmi Prasad	C-101	Block C	Owner
Arjun Mehta	C-102	Block C	Tenant
Fatima Begum	C-201	Block C	Owner

Q2. Residents with unpaid or partially paid bills and the balance pending (Presentation-II query)

Lists every resident whose flat has an unpaid or partly paid bill for a month that has already closed, together with the balance pending. It uses the view v_bill_balance joined with flat, block and resident, and orders the result by the largest balance. This is the query given during Presentation-II.

```

SELECT r.resident_name, b.block_name, f.flat_number, v.bill_month,
       v.status, v.balance AS pending_amount
FROM v_bill_balance v
JOIN flat f ON f.flat_id = v.flat_id
JOIN block b ON b.block_id = f.block_id
JOIN resident r ON r.flat_id = f.flat_id
WHERE v.status IN ('Unpaid', 'Partial')
      AND v.bill_month < '2026-10'
ORDER BY v.balance DESC, f.flat_number;

```

Table 8.2: Output of Query 2

resident_name	block_name	flat_number	bill_month	status	pending_amount
Anil Sharma	Block A	A-102	2026-08	Unpaid	5125.00
Anil Sharma	Block A	A-102	2026-09	Unpaid	5125.00
Fatima Begum	Block C	C-201	2026-09	Unpaid	5125.00
Arjun Mehta	Block C	C-102	2026-09	Unpaid	3575.00
Kavita Desai	Block B	B-201	2026-09	Partial	2637.50
Fatima Begum	Block C	C-201	2026-08	Partial	2637.50

Q3. Block-wise total amount collected

Totals the payments received for each block. Only payments are counted, so the figure is the money actually collected, and flats_paid shows how many flats in the block have made at least one payment.

```

SELECT b.block_name, COUNT(DISTINCT f.flat_id) AS flats_paid,
       SUM(p.amount_paid) AS total_collected
FROM payment p
JOIN maintenance_bill mb ON mb.bill_id = p.bill_id
JOIN flat f ON f.flat_id = mb.flat_id
JOIN block b ON b.block_id = f.block_id
GROUP BY b.block_name
ORDER BY total_collected DESC;

```

Table 8.3: Output of Query 3

block_name	flats_paid	total_collected
Block B	4	31762.50
Block A	3	20875.00
Block C	3	9662.50

Q4. Month-wise billed amount, collection and pending balance

Summarizes each billing month: the amount billed, the amount collected and the balance pending. The October balance is still open because the due date of 10 October has not yet passed.

```
SELECT bill_month, SUM(total_amount) AS total_billed,  
       SUM(paid_amount) AS collected, SUM(balance) AS pending  
FROM v_bill_balance  
GROUP BY bill_month  
ORDER BY bill_month;
```

Table 8.4: Output of Query 4

bill_month	total_billed	collected	pending
2026-08	37975.00	30212.50	7762.50
2026-09	38275.00	21812.50	16462.50
2026-10	37675.00	10275.00	27400.00

Q5. Flats that currently have no resident (LEFT JOIN with NULL check)

Finds flats that have no resident. A LEFT JOIN keeps every flat and the WHERE clause selects those with no matching resident.

```
SELECT b.block_name, f.flat_number, f.flat_type, f.area_sqft  
FROM flat f  
JOIN block b ON b.block_id = f.block_id  
LEFT JOIN resident r ON r.flat_id = f.flat_id  
WHERE r.resident_id IS NULL;
```

Table 8.5: Output of Query 5

block_name	flat_number	flat_type	area_sqft
Block C	C-202	2BHK	1050

Q6. Number of residents by resident type (GROUP BY)

Counts residents by type using GROUP BY.

```
SELECT resident_type, COUNT(*) AS total_residents  
FROM resident  
GROUP BY resident_type;
```

Table 8.6: Output of Query 6

resident_type	total_residents
Owner	8
Tenant	4

Q7. Payment-method-wise collection

Shows how much was collected through each payment method and the number of transactions.

```
SELECT payment_method, COUNT(*) AS transactions, SUM(amount_paid) AS amount
FROM payment
GROUP BY payment_method
ORDER BY amount DESC;
```

Table 8.7: Output of Query 7

payment_method	transactions	amount
UPI	6	18062.50
NetBanking	6	18062.50
Cash	5	15575.00
Card	4	10600.00

Q8. Flats whose total pending balance exceeds Rs. 5,000 (HAVING)

Uses HAVING to keep only those flats whose total pending balance across all months exceeds Rs. 5,000.

```
SELECT f.flat_number, SUM(v.balance) AS total_pending
FROM v_bill_balance v
JOIN flat f ON f.flat_id = v.flat_id
GROUP BY f.flat_id
HAVING SUM(v.balance) > 5000
ORDER BY total_pending DESC;
```

Table 8.8: Output of Query 8

flat_number	total_pending
A-102	15225.00
C-201	12737.50
B-201	7612.50
C-102	7000.00

Q9. Flats billed above the average bill amount (subquery)

Uses a subquery to compare the average bill of each flat with the average of all bills, returning flats billed above average.

```
SELECT f.flat_number, f.flat_type, ROUND(AVG(mb.total_amount), 2) AS avg_bill
FROM maintenance_bill mb
JOIN flat f ON f.flat_id = mb.flat_id
GROUP BY f.flat_id
HAVING AVG(mb.total_amount) > (SELECT AVG(total_amount) FROM maintenance_bill)
ORDER BY avg_bill DESC;
```

Table 8.9: Output of Query 9

flat_number	flat_type	avg_bill
A-102	3BHK	5075.00
C-201	3BHK	5075.00
B-201	3BHK	5025.00
C-102	2BHK	3475.00

Q10. Flats with all overdue bills cleared (NOT EXISTS)

Uses EXISTS and NOT EXISTS to list flats that have been billed and have no overdue bill that is still unpaid or partly paid.

```
SELECT b.block_name, f.flat_number
FROM flat f
JOIN block b ON b.block_id = f.block_id
WHERE EXISTS (SELECT 1 FROM maintenance_bill mb WHERE mb.flat_id = f.flat_id)
AND NOT EXISTS (SELECT 1 FROM maintenance_bill mb
                WHERE mb.flat_id = f.flat_id AND mb.status <> 'Paid' AND mb.bill_month < '2026-10')
ORDER BY b.block_name, f.flat_number;
```

Table 8.10: Output of Query 10

block_name	flat_number
Block A	A-101
Block A	A-201
Block A	A-202
Block B	B-101
Block B	B-102
Block B	B-202
Block C	C-101

9. UI DESIGN AND SCREENSHOTS

9.1 Design Approach

The user interface is a small web application intended for the society office. It follows a plain, uniform layout: a title bar, a single row of navigation links and a content area. Black text on a white background, bordered tables and standard form controls are used so that the screens are easy to read and print. Amounts are right-aligned and shown with two decimals, and every screen can be reached from the navigation bar in one click.

Table 9.1: Screens of the application

Screen	Route	Purpose
Dashboard	/	Totals of flats, residents, billed, collected and pending amounts, with a block-wise summary
Residents	/residents	Register of residents with block, flat, phone, type and move-in date
Bills	/bills	All bills with paid amount, balance and status; filter by month and status
Record Payment	/pay	Form to record a payment against an open bill
Defaulters	/defaulters	Residents with overdue unpaid or partly paid bills

9.2 Dashboard

The dashboard gives the committee an immediate view of the financial position. The figures are calculated from the view `v_bill_balance`.

Sunrise Heights Residency - Maintenance Billing System				
Dashboard	Residents	Bills	Record Payment	Defaulters
Dashboard				
Flats 12	Residents 12	Total billed (Rs.) 113,925.00	Collected (Rs.) 62,300.00	Pending (Rs.) 51,625.00
Block-wise Billing Summary				
Block	Flats billed	Billed (Rs.)	Collected (Rs.)	Pending (Rs.)
Block A	4	41,400.00	20,875.00	20,525.00
Block B	4	41,250.00	31,762.50	9,487.50
Block C	3	31,275.00	9,662.50	21,612.50

Figure 9.1: Dashboard screen

9.3 Resident Register

Sunrise Heights Residency - Maintenance Billing System

[Dashboard](#) [Residents](#) [Bills](#) [Record Payment](#) [Defaulters](#)

Resident Register

ID	Name	Block	Flat	Phone	Type	Move-in date
1	Ravi Kumar Reddy	Block A	A-101	9876510001	Owner	2018-06-01
2	Sunita Reddy	Block A	A-101	9876510002	Owner	2018-06-01
3	Anil Sharma	Block A	A-102	9876510003	Tenant	2024-02-15
4	Meera Iyer	Block A	A-201	9876510004	Owner	2019-03-20
5	Imran Khan	Block A	A-202	9876510005	Tenant	2025-07-01
6	Priya Nair	Block B	B-101	9876510006	Owner	2018-09-10
7	Suresh Babu	Block B	B-102	9876510007	Owner	2020-01-05
8	Kavita Desai	Block B	B-201	9876510008	Tenant	2023-11-12
9	Rahul Verma	Block B	B-202	9876510009	Owner	2019-05-30
10	Lakshmi Prasad	Block C	C-101	9876510010	Owner	2019-08-14
11	Arjun Mehta	Block C	C-102	9876510011	Tenant	2025-01-10
12	Fatima Begum	Block C	C-201	9876510012	Owner	2020-10-02

Figure 9.2: Resident register screen

9.4 Bills

The bills screen lists the bills of the selected month and status. Figure 9.3 shows the bills of September 2026.

Sunrise Heights Residency - Maintenance Billing System

[Dashboard](#) [Residents](#) [Bills](#) [Record Payment](#) [Defaulters](#)

Maintenance Bills

Month: 2026-09 Status: All Filter							
Bill	Flat	Month	Total (Rs.)	Paid (Rs.)	Balance (Rs.)	Due date	Status
12	A-101	2026-09	3,425.00	3,425.00	0.00	2026-09-10	Paid
13	A-102	2026-09	5,125.00	0.00	5,125.00	2026-09-10	Unpaid
14	A-201	2026-09	1,875.00	1,875.00	0.00	2026-09-10	Paid
15	A-202	2026-09	3,425.00	3,425.00	0.00	2026-09-10	Paid
16	B-101	2026-09	3,425.00	3,425.00	0.00	2026-09-10	Paid
17	B-102	2026-09	1,875.00	1,875.00	0.00	2026-09-10	Paid
18	B-201	2026-09	5,125.00	2,487.50	2,637.50	2026-09-10	Partial
19	B-202	2026-09	3,425.00	3,425.00	0.00	2026-09-10	Paid
20	C-101	2026-09	1,875.00	1,875.00	0.00	2026-09-10	Paid
21	C-102	2026-09	3,575.00	0.00	3,575.00	2026-09-10	Unpaid
22	C-201	2026-09	5,125.00	0.00	5,125.00	2026-09-10	Unpaid

Figure 9.3: Bills screen filtered for September 2026

9.5 Record Payment

The payment form lists only the bills that still have a balance. The office selects the bill, enters the amount, the method and the transaction reference and saves the payment.

Sunrise Heights Residency - Maintenance Billing System

[Dashboard](#) [Residents](#) [Bills](#) [Record Payment](#) [Defaulters](#)

Record Payment

Select bill

Bill 2 - Flat A-102 - 2026-08 - Balance Rs. 5,125.00

Amount paid (Rs.)

Payment method

UPI

Transaction reference

Save Payment

Figure 9.4: Record payment form

After the payment is saved, the screen confirms the amount and shows the new status of the bill, which is set by the database trigger. In Figure 9.5 the pending balance of Rs. 2,637.50 on the September bill of flat B-201 was paid in full and the bill became Paid.

Sunrise Heights Residency - Maintenance Billing System

[Dashboard](#) [Residents](#) [Bills](#) [Record Payment](#) [Defaulters](#)

Payment of Rs. 2,637.50 recorded for bill 18. Bill status: Paid.

Record Payment

Select bill

Bill 2 - Flat A-102 - 2026-08 - Balance Rs. 5,125.00

Amount paid (Rs.)

Payment method

UPI

Transaction reference

Save Payment

Figure 9.5: Confirmation after recording a payment

9.6 Defaulters

Sunrise Heights Residency - Maintenance Billing System

[Dashboard](#) [Residents](#) [Bills](#) [Record Payment](#) [Defaulters](#)

Defaulter List (overdue bills)

Resident	Block	Flat	Overdue bills	Pending (Rs.)
Anil Sharma	Block A	A-102	2	10,250.00
Fatima Begum	Block C	C-201	2	7,762.50
Arjun Mehta	Block C	C-102	1	3,575.00

Figure 9.6: Defaulter list screen

The screens in Figures 9.1 to 9.4 and 9.6 were captured from the sample database after the late fee had been applied. Figure 9.5 was captured after recording a payment through the form.

10. IMPLEMENTATION DETAILS

10.1 Technology Stack

Table 10.1: Technology stack

Layer	Technology	Role
Presentation	HTML 5, CSS	Pages, forms and tables displayed in the browser
Application	Python 3.12, Flask 3.1	Handles web requests, validates input and calls the database
Data access	Python sqlite3 (DB-API 2.0)	Opens connections, runs parameterized SQL and manages transactions
Database	SQLite 3.45	Stores the six tables and enforces constraints, triggers and the view

10.2 System Organization

The application follows a three-layer organization. The browser sends a request to a Flask route. The route runs one or more SQL statements through the sqlite3 module and receives rows. The rows are formatted as an HTML page and returned to the browser. Business rules that must always hold, such as the check that a payment does not exceed the bill and the update of the bill status, are placed in the database as constraints and triggers, not in the Python code. They therefore apply even when data is changed from the SQL prompt.

The files of the project are:

Table 10.2: Files of the project

File	Purpose
schema.sql	DDL: tables, indexes, triggers and view
seed.sql	DML: sample data (society, blocks, flats, residents, bills and payments)
build_db.py	Creates society.db from schema.sql and seed.sql
app.py	Flask application (routes and pages)
run_queries.py	Runs the ten queries and the DML demonstrations
test_cases.py	Executes the 19 test cases

10.3 Database Connectivity

A connection to the SQLite database is opened once per request and closed when the request ends. Two settings are important. Setting `row_factory` to `sqlite3.Row` lets the code read

columns by name, and the statement `PRAGMA foreign_keys = ON` switches on foreign key enforcement, which SQLite keeps off by default.

```
def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
        g.db.row_factory = sqlite3.Row
        g.db.execute("PRAGMA foreign_keys = ON")
    return g.db

@app.teardown_appcontext
def close_db(exc):
    db = g.pop("db", None)
    if db is not None:
        db.close()
```

All values supplied by the user are passed to SQL as parameters (the ? placeholders) and never joined into the SQL text. This prevents SQL injection and handles quotes and special characters correctly.

10.4 Key Code: Recording a Payment

The route below receives the payment form. It validates the amount, inserts a row into the payment table and commits. The database then runs the overpayment trigger and the status trigger. If the insert is rejected by a constraint or a trigger, the transaction is rolled back and the reason is shown to the user.

```
@app.route("/pay", methods=["GET", "POST"])
def pay():
    db = get_db()
    if request.method == "POST":
        try:
            bill_id = int(request.form["bill_id"])
            amount = float(request.form["amount"])
            method = request.form["method"]
            ref = request.form.get("ref", "").strip() or None
            if amount <= 0:
                raise ValueError("Amount must be greater than zero")
            db.execute("""INSERT INTO payment (bill_id, payment_date, amount_paid, payment_method,
transaction_ref)
VALUES (?, DATE('now'), ?, ?, ?)""", (bill_id, amount, method, ref))
            db.commit()
            st = db.execute("SELECT status FROM maintenance_bill WHERE bill_id = ?",
(bill_id,)).fetchone()["status"]
            flash(f"Payment of Rs. {amount:,.2f} recorded for bill {bill_id}. Bill status: {st}.")
        except (ValueError, sqlite3.Error) as e:
            db.rollback()
            flash(f"Payment not recorded: {e}")
    return redirect(url_for("pay"))
```

10.5 Key Code: Business Rules in the Database

The two triggers shown in Chapter 7 carry the main business rules. Before a payment is inserted, `trg_prevent_overpayment` adds the new amount to the amount already paid for the bill and aborts the insert if the result exceeds `total_amount`. After a payment is inserted, `trg_update_bill_status` compares the total paid with `total_amount` and sets the status to Paid when the bill is cleared or to Partial otherwise. The view `v_bill_balance` gives the paid amount and the balance of every bill, and is used by the dashboard, the bills screen, the defaulters screen and Queries 2, 4 and 8.

10.6 Running the Application

1. Install Python 3.12 and Flask (pip install flask).
2. Run `python build_db.py` to create the database `society.db` from `schema.sql` and `seed.sql`.
3. Run `python app.py` to start the web server and open the address shown in the terminal in a web browser.

11. TESTING

11.1 Test Approach

Testing was carried out at two levels. Test cases TC01 to TC15 test the database directly: valid and invalid inserts, every kind of constraint, the foreign key rules and the triggers. Test cases TC16 to TC19 test the web interface by sending requests to the Flask application. For every test the database was rebuilt from schema.sql and seed.sql so that each test started from the same known data. The tests were run by the script test_cases.py and the actual result of each test was recorded from the run.

11.2 Test Cases and Results

Table 11.1: Test cases and results

ID	Test description	Test data	Expected result	Actual result	Result
TC01	Insert a valid block	Block D, 5 floors, society 1	Row inserted	Row inserted	Pass
TC02	Duplicate block name within the same society	Block A, society 1	Rejected (UNIQUE)	UNIQUE constraint failed: block.society_id, block.block_name	Pass
TC03	Flat linked to a non-existent block	block_id = 99	Rejected (FOREIGN KEY)	FOREIGN KEY constraint failed	Pass
TC04	Invalid flat type	flat_type = 4BHK	Rejected (CHECK)	CHECK constraint failed: flat_type IN ('1BHK','2BHK','3BHK')	Pass
TC05	Negative flat area	area_sqft = -5	Rejected (CHECK)	CHECK constraint failed: area_sqft > 0	Pass
TC06	Duplicate resident phone number	phone = 9876510001	Rejected (UNIQUE)	UNIQUE constraint failed: resident.phone	Pass
TC07	Invalid resident type	resident_type = Guest	Rejected (CHECK)	CHECK constraint failed: resident_type IN ('Owner','Tenant')	Pass
TC08	Bill whose total does not equal the sum of its charges	total_amount = 9999	Rejected (CHECK)	CHECK constraint failed: total_amount = maintenance_amount + water_charge + parking_charge + late_fee	Pass
TC09	Second bill for the same flat and month	flat 1, 2026-08	Rejected (UNIQUE)	UNIQUE constraint failed: maintenance_bill.flat_id, maintenance_bill.bill_month	Pass
TC10	Part payment updates bill status (trigger)	Bill 25, Rs. 1,000	Status = Partial	Status = Partial	Pass
TC11	Final payment clears the bill (trigger)	Bill 25, balance Rs. 875.00	Status = Paid	Status = Paid	Pass
TC12	Overpayment against a bill	Bill 25, Rs. 99,999	Rejected (trigger)	Payment exceeds outstanding bill amount	Pass
TC13	Delete a block that still has flats	block_id = 1	Rejected (FOREIGN KEY)	FOREIGN KEY constraint failed	Pass

ID	Test description	Test data	Expected result	Actual result	Result
TC14	Duplicate transaction reference	ref = CAR202608004	Rejected (UNIQUE)	UNIQUE constraint failed: payment.transaction_ref	Pass
TC15	Unsupported payment method	method = Cheque	Rejected (CHECK)	CHECK constraint failed: payment_method IN ('Cash','UPI','Card','NetBanking')	Pass
TC16	All five web pages load	GET /, /residents, /bills, /pay, /defaulters	HTTP 200 for each	HTTP 200, 200, 200, 200, 200	Pass
TC17	Record a valid part payment from the form	Bill 25, Rs. 1,000, UPI	Success message, status Partial	Success message, status Partial	Pass
TC18	Reject a negative amount from the form	Bill 25, Rs. -50	Error message, no row saved	Error message shown	Pass
TC19	Bill filter by month and status	month = 2026-09, status = Unpaid	Only unpaid September bills listed	Only unpaid September bills listed	Pass

11.3 Summary of Results

A total of 19 test cases were executed and 19 passed, with no failures. The tests confirm that invalid data is rejected at the database level (unique, check and foreign key constraints), that part payments and final payments update the bill status correctly, that overpayment is refused, and that the web pages load and handle valid and invalid input as expected.

12. CONCLUSION AND FUTURE ENHANCEMENTS

12.1 Conclusion

The Residential Society Resident and Maintenance Billing System replaces scattered paper records with a single relational database. The design moved from an ER diagram to a normalized relational schema in third normal form, and the schema was implemented with keys, constraints, indexes, triggers and a view. The sample database supports all the common reporting needs of a society office, including resident registers, defaulter lists, block-wise and month-wise collections, and the identification of vacant flats. The plain web interface allows the office staff to use the system without knowing SQL. Nineteen test cases were executed and all passed. The project gave practical experience of ER modelling, normalization, SQL, integrity constraints, triggers and database connectivity.

12.2 Future Enhancements

1. Add user login with roles, such as administrator, accountant and resident, so that each person sees only the data they are permitted to see.
2. Add a resident portal where residents can view their bills and payment history.
3. Integrate an online payment gateway for UPI and card payments, with automatic recording of the transaction reference.
4. Send automatic SMS or e-mail reminders before the due date and when a bill becomes overdue.
5. Generate all monthly bills for the society with one command and apply late fees automatically on a schedule.
6. Generate printable receipts and monthly statements in PDF format.
7. Support complaints, visitor management and facility booking as additional modules.
8. Move the database to a client-server system such as MySQL or PostgreSQL for use by several office users at the same time.

REFERENCES

- [1]R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.
- [2]A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [3]E. F. Codd, “A relational model of data for large shared data banks,” Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.
- [4]P. P.-S. Chen, “The entity-relationship model—Toward a unified view of data,” ACM Trans. Database Syst., vol. 1, no. 1, pp. 9–36, Mar. 1976.
- [5]E. F. Codd, “Further normalization of the data base relational model,” in Data Base Systems, Courant Computer Science Symposia 6, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.
- [6]SQLite Consortium, “SQLite documentation.” [Online]. Available: <https://www.sqlite.org/docs.html>. [Accessed: Oct. 4, 2026].
- [7]SQLite Consortium, “SQLite foreign key support.” [Online]. Available: <https://www.sqlite.org/foreignkeys.html>. [Accessed: Oct. 4, 2026].
- [8]SQLite Consortium, “CREATE TRIGGER.” [Online]. Available: https://www.sqlite.org/lang_createttrigger.html. [Accessed: Oct. 4, 2026].
- [9]Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases.” [Online]. Available: <https://docs.python.org/3/library/sqlite3.html>. [Accessed: Oct. 4, 2026].
- [10]Pallets Projects, “Flask documentation.” [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: Oct. 4, 2026].
- [11]Oracle Corporation, “MySQL 8.0 reference manual.” [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: Oct. 4, 2026].

APPENDIX: GITHUB REPOSITORY LINK

The complete source code of the project, including the database scripts, the web application and the test script, is available in the following GitHub repository:

<https://github.com/nullvectorcodes/<repository-name>>

The repository contains the following files:

Table A.1: Contents of the repository

File	Description
schema.sql	DDL statements: tables, constraints, indexes, triggers and view
seed.sql	DML statements: sample data for the society
build_db.py	Script that builds the database from the SQL files
app.py	Flask web application
run_queries.py	Script that runs the queries listed in Chapter 8
test_cases.py	Script that executes the test cases listed in Chapter 11